DAY 25 OF 30

Ship it safely

Git integration and deployment pipelines.
Changes that do not break production.

Dev

Test

Prod

Keshav Maheshwari

Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

THE PROBLEM

"I just changed one thing" in production.

1	**The change**	Made directly in the live workspace
2	**The test**	The CEO opening the report
3	**The undo button**	Does not exist

Without versions and stages, **every edit is a live experiment on real users**.

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

THE SIMPLE VERSION

Rehearsal before the live show.

No theatre opens a play without rehearsing it. Mistakes happen on an empty stage, not in front of the audience.

Deployment stages are your **rehearsal rooms**, and Git is the **script with every revision kept**.

✓ **Git**	Every change saved, with history
✓ **Dev and Test**	Where mistakes are cheap
✓ **Production**	Only what passed

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

HOW IT WORKS

The path of a change

1. Connect the workspace to a **Git branch**
2. Work in **Dev**, commit, and open a pull request
3. **Deploy** to Test and check the result
4. **Deploy** to Production when it passes
5. **Undo** a bad change by reverting the commit in Git

Two tools, two jobs

Git keeps the history of your items. Deployment pipelines move them between stages.

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

FOR ENGINEERS

What to set up

Git providers

Azure DevOps, GitHub and GitHub Enterprise are supported, in their cloud versions.

Stages are flexible

A deployment pipeline can have from 2 to 10 stages. Three is the default.

Variable libraries

Keep values that differ per stage, such as connection names, out of the items themselves.

Automate when ready

The fabric-cicd library and the Fabric CLI let a build server deploy for you.

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

THE HONEST PART

How much process do you need?

MINIMUM FOR EVERYONE

Separate Dev and Prod workspaces

Everything committed to Git

No direct edits in production

ADD WHEN YOU GROW

Automated deployments from a build server

Pull request reviews for each change

Automated tests on deployment

> **The catch:** deployment moves definitions, not data. A lakehouse arrives in Test with its structure and no rows. Plan how each stage gets its data.

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

WATCH OUT

Four things to plan for

1. **Item support varies**: check each item type you use
2. **Stage-specific values** such as connections and IDs
3. **Data in lower stages**: sample, shortcut or reload
4. **Who may deploy** to production, and who approves

The goal is boring releases. **Boring is good.**

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

SAVE THIS

Shipping in five lines

1. **Git** keeps every version of every item
2. **Stages** keep mistakes away from users
3. **Variables** handle per-stage differences
4. **Definitions move**, data does not
5. **Nobody edits** production by hand

Tomorrow: Monitoring.
Knowing something is wrong before the users do.

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Follow for Day 26